

## TP Sheet 5 - Motion planning

Prepared by: Gonalo Leo

With contributions by: Professor Lus Paulo Reis and Professor Armando Sousa

Be sure to check the theoretical lecture on “Motion planning” for an in-depth explanation of all the algorithms in this sheet.

### **Exercise 1 - Grid-based search using the Dijkstra/A\* algorithm**

Consider the simulation scenario in the dots.wbt or obstacles.wbt Webots files, with the usual e-puck robot, equipped with a LiDAR sensor. Remember that these files were generated using the create\_map.py script and thus the walls are detectable by the LiDAR but the robot can traverse them and no collisions occur (think of them as “holograms”).

Write a robot controller that moves the robot from its current position to a final/goal position given by the final\_position vector in the main function, without colliding with obstacles, using the A\* algorithm. Assume that the robot’s initial pose is known alongside the position of all the obstacles (which are read from a .csv file in the main function).

The A\* algorithm will be run in a graph built using a grid-based search. The class representing the graph is MetricGraph (from metric\_graph.py). It derives from a more generic Graph class (in graph.py), composed of instances of Vertex and Edge (from vertex\_edge.py).

The MutablePriorityQueue class (in mutable\_priority\_queue.py) is already 100% complete and ready to be used in the A\* algorithm.

In this exercise, you will fill section of the Graph class to run the A\* algorithm and determine the shortest path from A to B.

The ex1.py file, the exercise’s main file, already contains all the code needed for the robot to move along A\*’s computed path. It already includes an heuristic cost function based on the Euclidean distance between nodes.

The utils.py file, already has all the functions needed for collision detection, namely the collides\_segment\_rectangle function.

The code will generate a plot of the graph, where green vertices are part of the path, blue vertices were processed by A\*/Dijkstra but are not part of the path and red vertices were not processed.

To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

1. In the Graph class, complete in the relax function, with the logic to update a vertex's "path", "dist" and "cost" fields when a new shorter path is found to that vertex.
2. In the Graph class, fill in the A\* algorithm:
  - a. Initialize the vertices, by setting the appropriate initial value for their "path" and "dist" fields.
  - b. Fill in the code to update the queue, when a new vertex is found and when a shorter path is found for it. You only need to make a call to the correct function of MutablePriorityQueue for each one.
3. In the Graph class, in the get\_path function, after adding the destination vertex, add the rest of the vertices to the path by following the "path" attribute of the vertices (which is set after calling A\*). Do not forget to reverse the path array at the end.
4. In ex1.py, in the create\_grid\_graph function:
  - a. Complete the generation of the grid's vertices by calling MetricGraph's add\_vertex method. For the color argument, set these vertices' color as 'blue'.
  - b. Connect the final point to the closest point in the grid using edges. The logic is very similar to the one for the initial point, which is already done.
  - c. Complete the code to connect each of the grid's vertices to their top neighbor. The logic is very similar to the one for the left neighbor, also already done.
5. In the Graph class, fill in the Dijkstra algorithm by simply making an appropriate call to A\*. Suggestion: use a lambda. You only need one line of code for this.
6. Test your solution in different scenarios. Check that A\* requires processing less vertices than Dijkstra (i.e. the former algorithm will have more vertices in red than the latter).

## **Exercise 2 - Rapidly Exploring Random Trees (RRT)**

You will now work with Rapidly Exploring Random Trees (RRT) for the same goal as exercise 1. This exercise requires you to have filled in the get\_path function from the Graph class (see exercise 1).

To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

1. In ex1.py, in the create\_rrt function, fill in all the logic for an iteration of RRT. Check the .py file for more in-depth tips of each step of an iteration.
2. Test your solution in different scenarios. Try changing the incremental\_distance parameter. What happens?

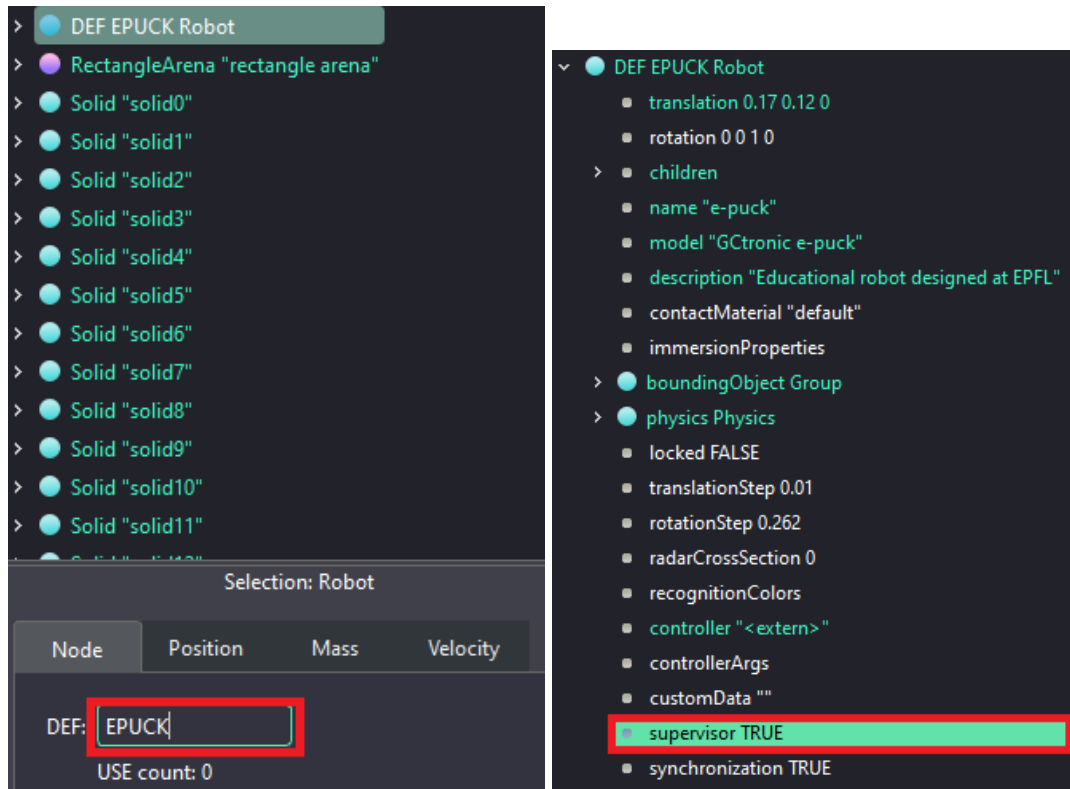
## **Exercise 3 - Artificial Potential Fields (APF)**

You will now work with Artificial Potential Fields (RRT) for the same goal as exercise 1. You are highly recommended to explore mostly using the dots.wbt world file. In the other files, the robot is very prone to local minima and has a lot of trouble reaching the goal.

To ex3.py file already contains all the code needed to draw quiver plots, which help you visualize the vectors of the attractive, repulsive and resulting forces on a set of points in a

grid of the map. To compute the repulsive forces, which require a LiDAR scan, the code uses a Supervisor robot, which is a “special” robot with extra privileges that allows for the programmer to change its position in run-time. To work with a supervisor, you need to configure the robot correctly in the Webots file (as seen in the images below):

- ensure the robot’s DEF field is not empty. For example, add “EPUCK” to it, so you can reference the supervisor in the code (left image).
- ensure the checkbox for the “supervisor” attribute is set to TRUE (right image).



To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

1. In compute\_attractive\_force function, compute and return the attractive force.
2. In compute\_repulsive\_force function, compute and return the repulsive force:
  - a. Complete the code that find the LiDAR readings that are local minima, according to the size of the sliding window (window\_size parameter).
  - b. Complete the computation of the repulsive force produced by each obstacle.
3. In the compute\_resulting\_force function, compute and return the resulting force, which combines the attractive and repulsive forces.
4. Test your solution in different scenarios. Try changing the d\_max parameter. What happens?